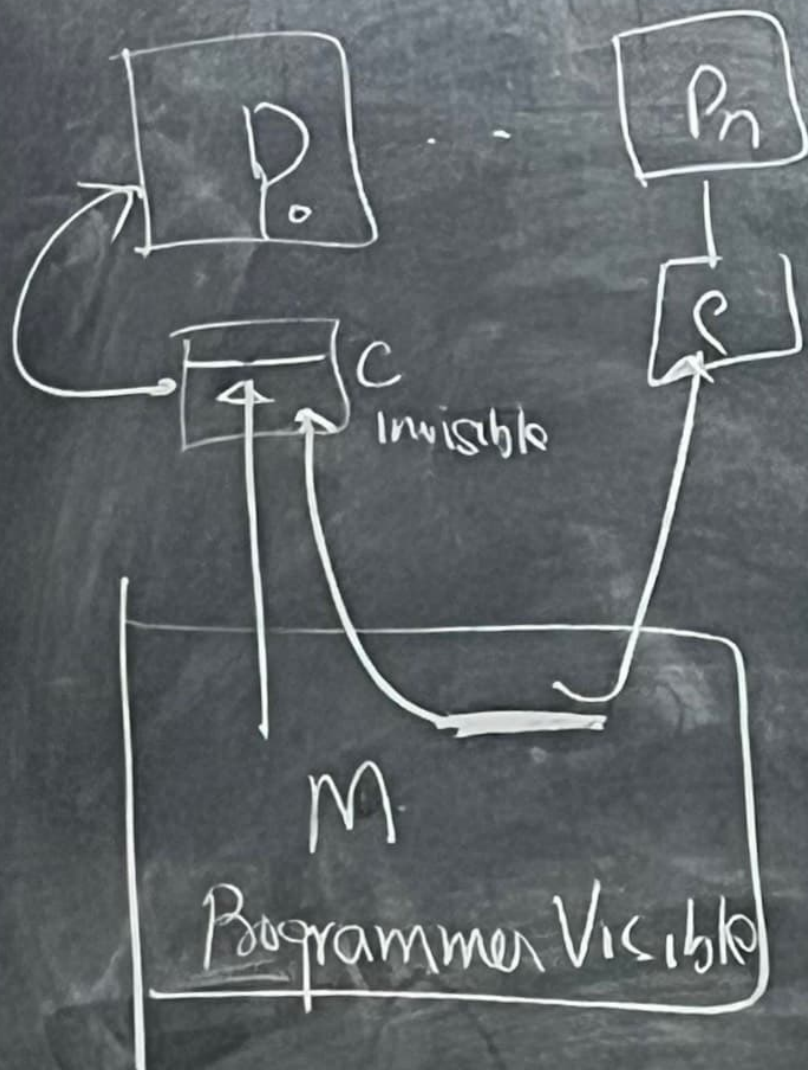


Difficulty in Parallel Programming

- Little difficulty if parallelism is natural
 - “Embarrassingly parallel” applications
 - Multimedia, physical simulation, graphics
 - Large web servers, databases?
- Difficulty is in
 - Getting parallel programs to work correctly
 - Optimizing performance in the presence of bottlenecks
- Much of **parallel computer architecture** is about
 - Designing machines that overcome the sequential and parallel bottlenecks to achieve higher performance and efficiency

Caching in Multiprocessors

- Caching not only complicates ordering of **all operations**...
 - A memory location can be present in multiple caches
 - Prevents the effect of a store or load to be seen by other processors → makes it difficult for all processors to see the same global order of (all) memory operations
- ... but it also complicates ordering of **operations on a single memory location**
 - A single memory location can be present in multiple caches
 - Makes it difficult for processors that have cached the same location to have the correct value of that location (in the presence of updates to that location)



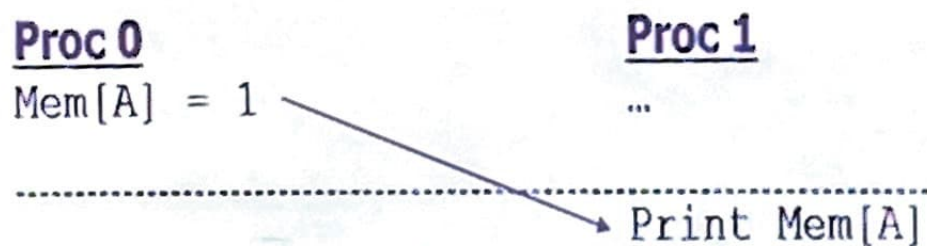
Memory Consistency vs. Cache Coherence

- Consistency is about ordering of all memory operations from different processors (i.e., to different memory locations)

Cache Coherence

Shared Memory Model

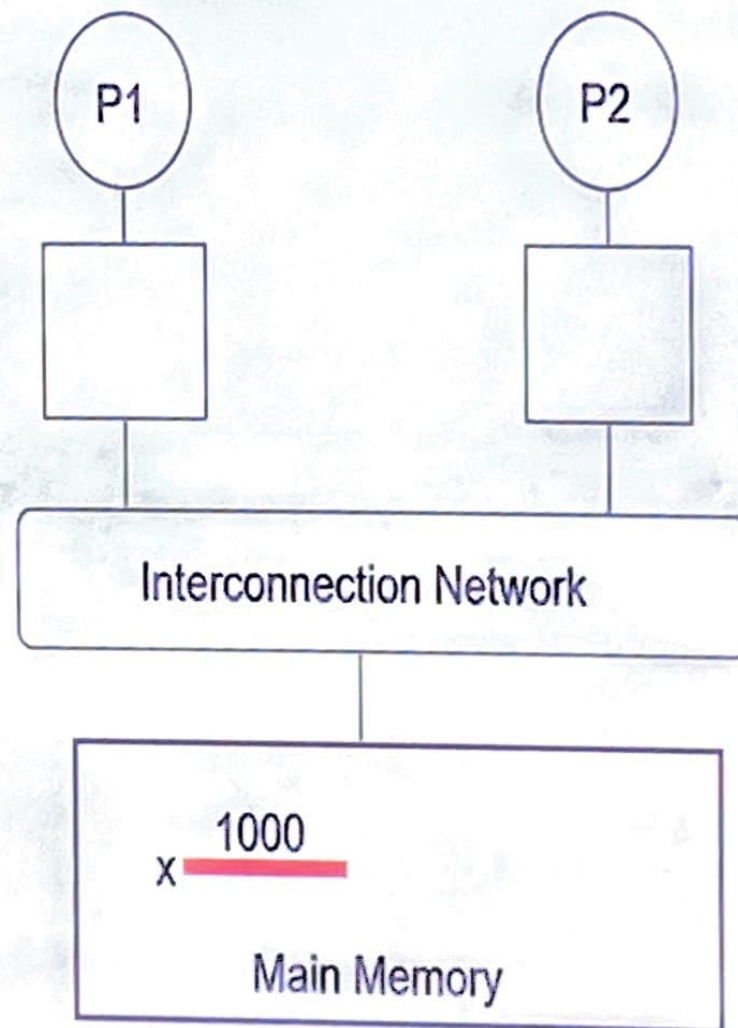
- Many parallel programs communicate through shared memory
- Proc 0 writes to an address, followed by Proc 1 reading
 - This implies communication between the two



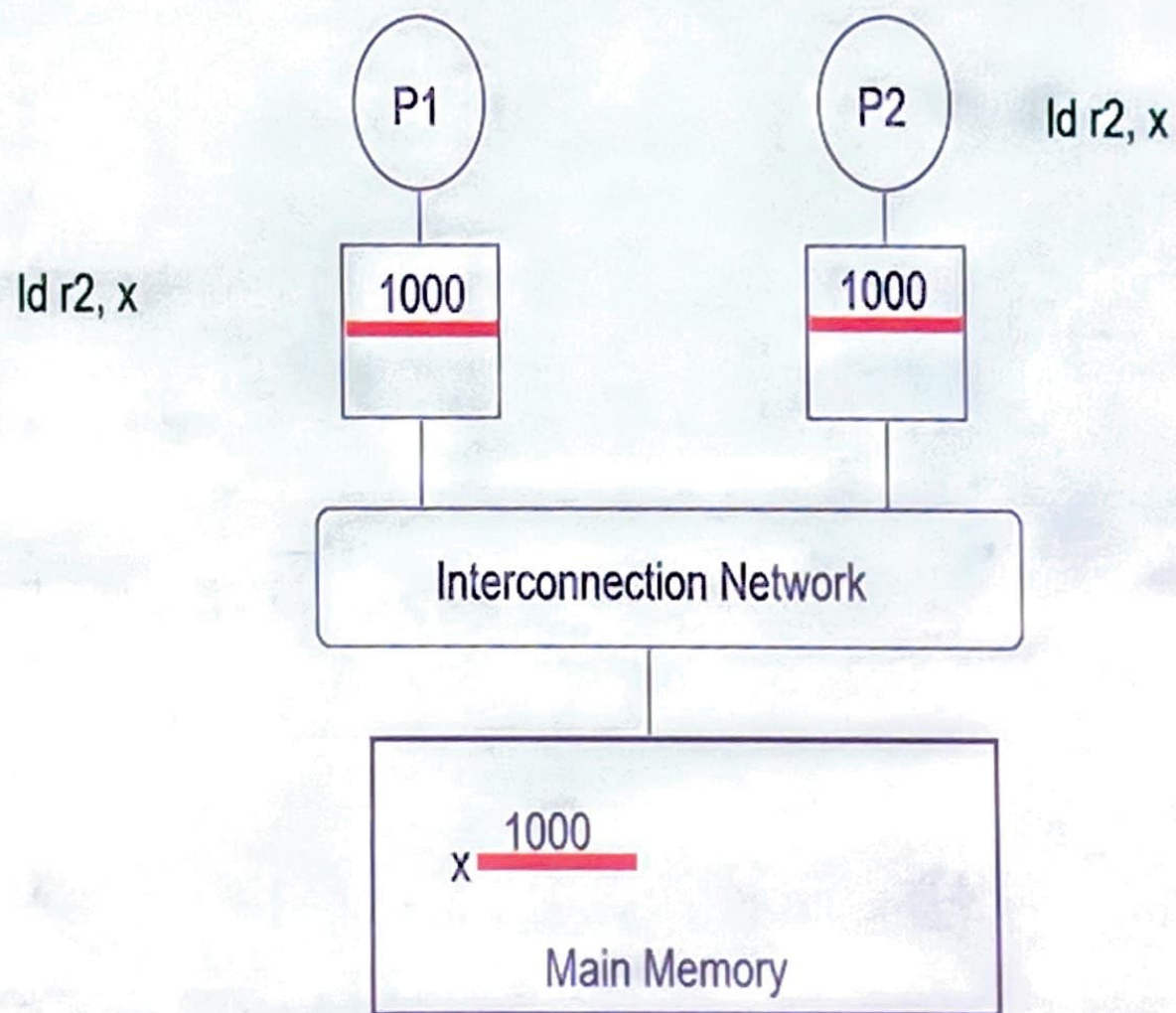
- Each read should receive the value last written by anyone
 - This requires synchronization (what does last written mean?)
- What if Mem[A] is cached (at either end)?

Cache Coherence

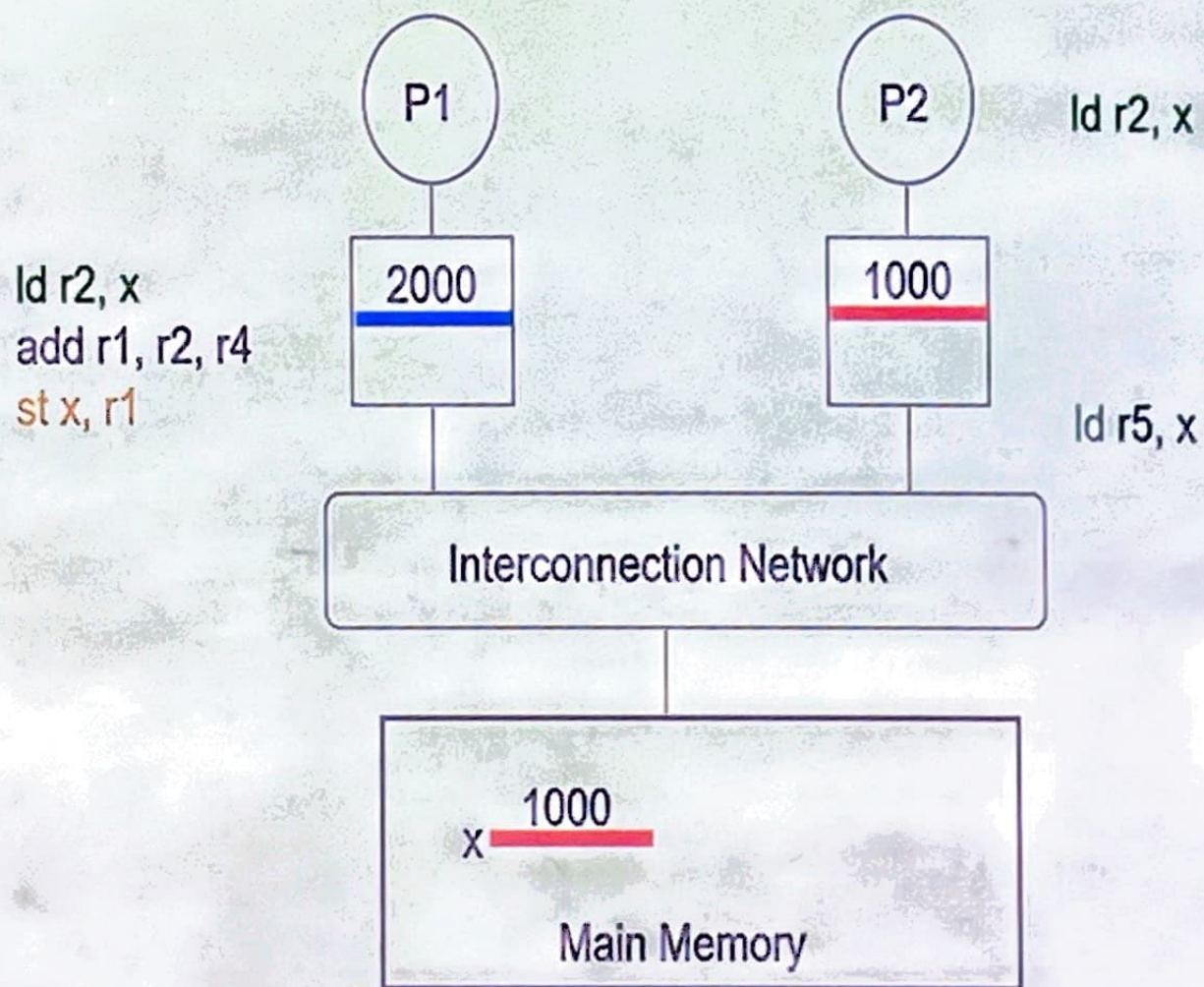
- Basic question: If multiple processors cache the same block, how do they ensure they all see a consistent state?



The Cache Coherence Problem



The Cache Coherence Problem



Difficulty in Parallel Programming

- Little difficulty if parallelism is natural
 - "Embarrassingly parallel" applications
 - Multimedia, physical simulation, graphics
 - Large web servers, databases?
- Difficulty is in
 - Getting parallel programs to work correctly
 - Optimizing performance in the presence of bottlenecks
- Much of **parallel computer architecture** is about
 - Designing machines that overcome the sequential and parallel bottlenecks to achieve higher performance and efficiency
 - Making programmer's job easier in writing correct and high-performance parallel programs


```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main() {
    const long N = 100000000; // 100 million elements
    double *arr = (double*) malloc(N * sizeof(double));
    double serial_sum = 0.0, parallel_sum = 0.0;
    double start, end;

    // Initialize array
    for (long i = 0; i < N; i++)
        arr[i] = 1.0;

    // ---- Serial version ----
    start = omp_get_wtime();
    for (long i = 0; i < N; i++)
        serial_sum += arr[i];
    end = omp_get_wtime();
    printf("Serial sum = %.0f, time = %.3f sec\n", serial_sum, end - start);
    double serialTime= end-start;

    // ---- Parallel version ----
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:parallel_sum)
    for (long i = 0; i < N; i++)
        parallel_sum += arr[i];
    end = omp_get_wtime();
    printf("Parallel sum = %.0f, time = %.3f sec\n", parallel_sum, end - start);
```

```
double serial_sum = 0.0, parallel_sum = 0.0;  
double start, end;
```

```
// Initialize array  
for (long i = 0; i < N; i++)  
    arr[i] = 1.0;
```

```
// ---- Serial version ----  
start = omp_get_wtime();  
for (long i = 0; i < N; i++)  
    serial_sum += arr[i];  
end = omp_get_wtime();  
printf("Serial sum = %.0f, time = %.3f sec\n", serial_sum, end - start);  
double serialTime= end-start;
```

```
// ---- Parallel version ----  
start = omp_get_wtime();  
#pragma omp parallel for reduction(+:parallel_sum)  
for (long i = 0; i < N; i++)  
    parallel_sum += arr[i];  
end = omp_get_wtime();  
printf("Parallel sum = %.0f, time = %.3f sec\n", parallel_sum, end - start);
```

```
int num_procs = omp_get_num_procs();  
printf("Number of logical CPUs available: %d\n", num_procs);
```

```
double parallelTime= end-start;
```

```
printf("Speedup = %.2fx\n", serialTime/parallelTime);  
free(arr);
```


691 updates can be applied immediately.
445 of these updates are standard security updates.
To see these additional updates run: `apt list --upgradable`

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.

To check for new updates run: `sudo apt update`

Failed to connect to <https://changelogs.ubuntu.com/meta-release-lts>. Check your Internet connection or proxy settings

Last login: Wed Oct 8 08:07:18 2025 from 10.237.21.10

```
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$ cd col7001/  
(base) kolin@mosaic:~/col7001$ cd parallel/  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.c  
(base) kolin@mosaic:~/col7001/parallel$ g
```

The list of available updates is more than a week old.

To check for new updates run: `sudo apt update`

Failed to connect to <https://changelogs.ubuntu.com/meta-release-lts>. Check your Internet connection or proxy settings

Last login: Wed Oct 8 08:07:18 2025 from 10.237.21.10

```
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$ cd col7001/  
(base) kolin@mosaic:~/col7001$ cd parallel/  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.c  
(base) kolin@mosaic:~/col7001/parallel$ gcc speedupDemo.c -fopenmp  
(base) kolin@mosaic:~/col7001/parallel$ ./a.out  
Serial sum = 100000000, time = 0.242 sec  
Parallel sum = 100000000, time = 0.016 sec  
Number of logical CPUs available: 32  
Speedup = 14.97x  
(base) kolin@mosaic:~/col7001/parallel$
```



```
import multiprocessing as mp
import time
import numpy as np
```

```
def partial_sum(data_slice):
    return np.sum(data_slice)
```

```
if __name__ == "__main__":
    N = 100_000_000 # 100 million
    arr = np.ones(N, dtype=np.float64)
```

```
    num_cpus = mp.cpu_count()
    print(f"Number of logical CPUs available: {num_cpus}")
```

```
    # ---- Serial version ----
```

```
    start = time.time()
    serial_sum = np.sum(arr)
    serial_time = time.time() - start
    print(f"Serial sum = {serial_sum:.0f}, time = {serial_time:.3f} sec")
```

```
    # ---- Parallel version ----
```

```
    num_workers = num_cpus # use all available CPUs
    chunk_size = N // num_workers
    chunks = [arr[i*chunk_size:(i+1)*chunk_size] for i in range(num_workers)]
```

```
    start = time.time()
    with mp.Pool(processes=num_workers) as pool:
        results = pool.map(partial_sum, chunks)
    parallel_sum = sum(results)
    parallel_time = time.time() - start
```

```
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$ cd col7001/  
(base) kolin@mosaic:~/col7001$ cd parallel/  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.c  
(base) kolin@mosaic:~/col7001/parallel$ gcc speedupDemo.c -fopenmp  
(base) kolin@mosaic:~/col7001/parallel$ ./a.out  
Serial sum = 100000000, time = 0.242 sec  
Parallel sum = 100000000, time = 0.016 sec  
Number of logical CPUs available: 32  
Speedup = 14.97x  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.py  
(base) kolin@mosaic:~/col7001/parallel$ python3 speedupDemo.py  
Number of logical CPUs available: 32  
Serial sum = 100000000, time = 0.057 sec  
Parallel sum = 100000000, time = 2.770 sec  
Speedup = 0.02x  
(base) kolin@mosaic:~/col7001/parallel$ python3 speedupDemo-v2.py  
Using 32 CPUs  
Serial time: 0.468s  
Parallel time: 0.214s  
Speedup: 2.19x  
(base) kolin@mosaic:~/col7001/parallel$
```



```
import concurrent.futures, numpy as np, time, os
```

```
def partial_sum(start, end):  
    # Generate and compute locally (no data transfer)  
    arr = np.arange(start, end, dtype=np.float64)  
    return np.sum(np.sqrt(arr * 3.14159))
```

```
if __name__ == "__main__":  
    N = 100_000_000  
    num_cpus = os.cpu_count()  
    chunk_size = N // num_cpus
```

```
    print(f"Using {num_cpus} CPUs")
```

```
    # Serial
```

```
    start = time.time()  
    total_serial = 0  
    for i in range(num_cpus):  
        total_serial += partial_sum(i*chunk_size, (i+1)*chunk_size)  
    serial_time = time.time() - start
```

```
    # Parallel
```

```
    start = time.time()  
    with concurrent.futures.ProcessPoolExecutor(max_workers=num_cpus) as ex:  
        results = list(ex.map(partial_sum,  
                               [i*chunk_size for i in range(num_cpus)],  
                               [(i+1)*chunk_size for i in range(num_cpus)]))  
    parallel_time = time.time() - start
```

```
    print(f"Serial time: {serial_time:.3f}s")
```

```
speedupDemo-v2.py
```

```
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$  
(base) kolin@mosaic:~$ cd col7001/  
(base) kolin@mosaic:~/col7001$ cd parallel/  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.c  
(base) kolin@mosaic:~/col7001/parallel$ gcc speedupDemo.c -fopenmp  
(base) kolin@mosaic:~/col7001/parallel$ ./a.out  
Serial sum = 100000000, time = 0.242 sec  
Parallel sum = 100000000, time = 0.016 sec  
Number of logical CPUs available: 32  
Speedup = 14.97x  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo.py  
(base) kolin@mosaic:~/col7001/parallel$ python3 speedupDemo.py  
Number of logical CPUs available: 32  
Serial sum = 100000000, time = 0.057 sec  
Parallel sum = 100000000, time = 2.770 sec  
Speedup = 0.02x  
(base) kolin@mosaic:~/col7001/parallel$ python3 speedupDemo-v2.py  
Using 32 CPUs  
Serial time: 0.468s  
Parallel time: 0.214s  
Speedup: 2.19x  
(base) kolin@mosaic:~/col7001/parallel$ less speedupDemo-v2.py  
(base) kolin@mosaic:~/col7001/parallel$
```


$\text{fib}(n)$

$f(n-1)$

$f(n-2)$

$f(n-2)$

$f(n-3)$

$f(n-3)$

$f(n-4)$

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
// Recursive Fibonacci with OpenMP tasks
```

```
long fib(int n) {
```

```
    long x, y;
```

```
    if (n < 2) return n;
```

```
    #pragma omp task shared(x)
```

```
    x = fib(n - 1);
```

```
    #pragma omp task shared(y)
```

```
    y = fib(n - 2);
```

```
    #pragma omp taskwait
```

```
    return x + y;
```

```
}
```

```
int main() {
```

```
    int n = 30;
```

```
    long result;
```

```
    double start, end;
```

```
    printf("Number of logical CPUs available: %d\n", omp_get_num_procs());
```

```
    // --- Serial computation ---
```

```
    start = omp_get_wtime();
```

```
    long serial_result = 0;
```

```
    for (int i = 0; i < 1; i++) // single run
```

```
fibonacci.c
```


(base) kolin@mosaic:~/col7001/parallel\$
(base) kolin@mosaic:~/col7001/parallel\$
(base) kolin@mosaic:~/col7001/parallel\$ less fibonacci.c
(base) kolin@mosaic:~/col7001/parallel\$ gcc fibonacci.c -fopenmp
(base) kolin@mosaic:~/col7001/parallel\$./a.out

Number of logical CPUs available: 32

Serial Fibonacci(30) = 832040, time = 0.069 sec

Parallel Fibonacci(30) = 832040, time = 4.225 sec

Speedup = 0.02x

(base) kolin@mosaic:~/col7001/parallel\$

```
long x, y;

if (n < 2) return n;

#pragma omp task shared(x)
x = fib(n - 1);

#pragma omp task shared(y)
y = fib(n - 2);

#pragma omp taskwait
return x + y;
}

int main() {
    int n = 30;
    long result;
    double start, end;

    printf("Number of logical CPUs available: %d\n", omp_get_num_procs());

    // --- Serial computation ---
    start = omp_get_wtime();
    long serial_result = 0;
    for (int i = 0; i < 1; i++) // single run
        serial_result = fib(n);
    end = omp_get_wtime();
    double serial_time = end - start;

    printf("Serial Fibonacci(%d) = %ld, time = %.3f sec\n", n, serial_result, serial_time);
```



```
#include <stdio.h>
#include <omp.h>

long fib(int n) {
    if (n < 2) return n;

    // Base case: small problems - compute serially
    if (n < 25)
        return fib(n - 1) + fib(n - 2);

    long x, y;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() {
    int n = 40;
    long result;
    double t1, t2;

    printf("Using %d CPUs\n", omp_get_num_procs());

    // --- Serial version ---
    fibonacci-v2.c
```


h or --help Print Help (this message) and exit
--version Print version information and exit

Arguments recognised by gvim (GTK+ version):

-background <color> Use <color> for the background (also: -bg)
-foreground <color> Use <color> for normal text (also: -fg)
-font Use for normal text (also: -fn)
-geometry <geom> Use <geom> for initial geometry (also: -geom)
-iconic Start Vim iconified
-reverse Use reverse video (also: -rv)
-display <display> Run Vim on <display> (also: --display)
--role <role> Set a unique role to identify the main window
--socketid <xid> Open Vim inside another GTK widget
--echo-wid Make gvim echo the Window ID on stdout

```
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less fibonacci.c  
(base) kolin@mosaic:~/col7001/parallel$ less fibonacci-v2.c  
(base) kolin@mosaic:~/col7001/parallel$ gcc fibonacci-v2.c -fopenmp  
(base) kolin@mosaic:~/col7001/parallel$ ./a.out
```

Using 32 CPUs

Serial Fibonacci(40) = 102334155, time = 0.918 sec

Parallel Fibonacci(40) = 102334155, time = 0.105 sec

Speedup = 8.78x

```
(base) kolin@mosaic:~/col7001/parallel$
```



```
import concurrent.futures
import os
import time
```

```
# Recursive Fibonacci with limited parallelism
```

```
def fib(n):
```

```
    if n < 2:
        return n
```

```
    # For small n, recursion overhead dominates - compute serially
```

```
    if n < 20:
        return fib(n - 1) + fib(n - 2)
```

```
    # For larger n, run in parallel
```

```
    with concurrent.futures.ProcessPoolExecutor(max_workers=2) as executor:
```

```
        f1 = executor.submit(fib, n - 1)
```

```
        f2 = executor.submit(fib, n - 2)
```

```
        return f1.result() + f2.result()
```

```
def fib_serial(n):
```

```
    if n < 2:
        return n
```

```
    return fib_serial(n - 1) + fib_serial(n - 2)
```

```
if __name__ == "__main__":
```

```
    n = 40 # reasonable runtime
```

```
    print(f"Number of logical CPUs available: {os.cpu_count()}")
```

fibonacci.py

-foreground <color> Use <color> for normal text (also: -fg)
-font Use for normal text (also: -fn)
-geometry <geom> Use <geom> for initial geometry (also: -geom)
-iconic Start Vim iconified
-reverse Use reverse video (also: -rv)
-display <display> Run Vim on <display> (also: --display)
--role <role> Set a unique role to identify the main window
--socketid <xid> Open Vim inside another GTK widget
--echo-wid Make gvim echo the Window ID on stdout

```
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$  
(base) kolin@mosaic:~/col7001/parallel$ less fibonacci.c  
(base) kolin@mosaic:~/col7001/parallel$ less fibonacci-v2.c  
(base) kolin@mosaic:~/col7001/parallel$ gcc fibonacci-v2.c -fopenmp  
(base) kolin@mosaic:~/col7001/parallel$ ./a.out
```

Using 32 CPUs

Serial Fibonacci(40) = 102334155, time = 0.918 sec

Parallel Fibonacci(40) = 102334155, time = 0.105 sec

Speedup = 8.78x

```
(base) kolin@mosaic:~/col7001/parallel$ less fibonacci.py
```

```
(base) kolin@mosaic:~/col7001/parallel$ python3 fibonacci.py
```

Number of logical CPUs available: 32

Serial Fibonacci(40) = 102334155, time = 13.569 sec

Parallel Fibonacci(40) = 102334155, time = 16.156 sec

Speedup = 0.84x

```
(base) kolin@mosaic:~/col7001/parallel$
```



```
#include <stdio.h>
#include <omp.h>
```

```
long fib(int n) {
    if (n < 2) return n;

    // Base case: small problems - compute serially
    if (n < 25)
        return fib(n - 1) + fib(n - 2);

    long x, y;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

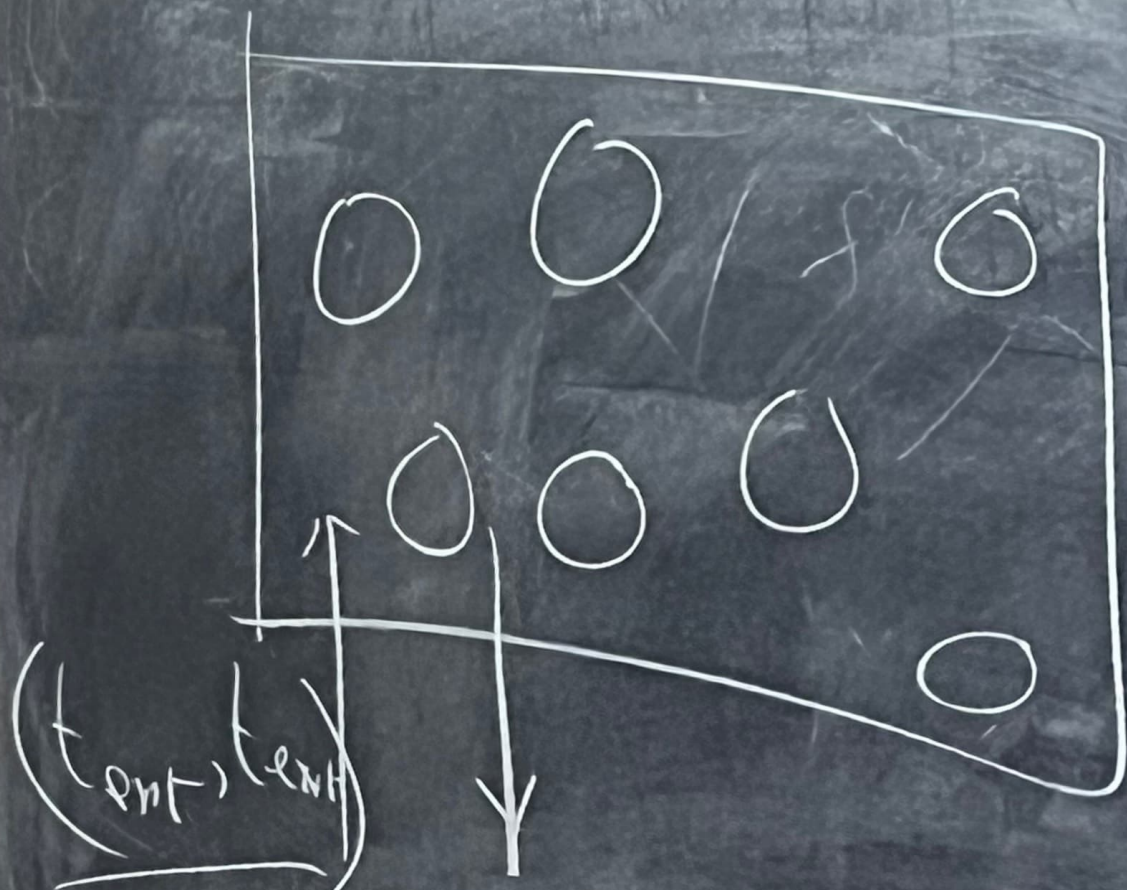
    #pragma omp taskwait
    return x + y;
}
```

```
int main() {
    int n = 40;
    long result;
    double t1, t2;

    printf("Using %d CPUs\n", omp_get_num_procs());

    // --- Serial version ---
```

```
fibonacci-v2.py
```



$(1, 3)$ $(2, 9)$ $(5, 6)$ $(7, 8)$